

Krypteia MCP — Architecture and Security Review

Version: 0.1.0 | **Author:** Anna Hix | **Date:** March 18, 2026

Repository: github.com/annawhooo/krypteia-mcp

Classification: Internal — For Security Review

1. Executive summary

Krypteia is an open-source Python MCP (Model Context Protocol) server that provides encrypted credential storage and authenticated request proxying for LLM agents. It enables AI assistants like Claude to make authenticated HTTP requests and access login-protected web portals without the credential ever appearing in the LLM's conversation context.

Named for Sparta's covert operatives (Krypteia) — the Greek root of "cryptography."

Key security properties:

- Credentials encrypted at rest with AES-256-GCM (per-entry unique nonces)
- Master key stored in OS keyring (Windows Credential Manager / macOS Keychain / Linux Secret Service)
- URL allowlisting prevents credential use against unauthorized endpoints (fail-closed)
- Response sanitization scrubs leaked credentials before returning to the LLM
- Tamper-evident audit log with SHA-256 hash chain
- Credentials never cross into the LLM context window

2. Architecture overview

2.1 Trust boundaries

The system operates across four trust boundaries:

Boundary 1 — User-controlled (one-time setup). The user interacts with the Krypteia CLI to initialize the master key and add credentials. Plaintext credentials exist only during this interactive session (masked input via `getpass`). The master key is derived via PBKDF2-HMAC-SHA256 (600,000 iterations) and stored in the OS keyring.

Boundary 2 — Krypteia MCP server process (localhost, STDIO). The MCP server runs as a local subprocess spawned by Claude Desktop. It communicates via STDIO (stdin/stdout JSON-RPC). All credential resolution, HTTP request injection, browser automation, and response sanitization happen inside this process. The server never listens on a network port.

Boundary 3 — LLM context (Claude Desktop). Claude Desktop communicates with Krypteia via MCP tool calls over STDIO. Claude sends tool invocations (alias, URL, method) and receives sanitized results. Claude never receives passwords, API keys, tokens, session cookies, or usernames.

Boundary 4 — External targets (REST APIs, web portals). Authenticated requests are made from the Krypteia server process to external targets. Credentials are injected into HTTP headers (for API proxy) or browser form fields (for web login). TLS handles in-transit encryption to the target.

2.2 Components

Component	Role	Technology
Encrypted store	Per-entry AES-256-GCM encryption	Python cryptography library
Keychain	Master key from OS keyring	Python keyring library
URL allowlist	Per-credential URL pattern enforcement	fnmatch glob matching
Response sanitizer	Scrubs leaked creds from responses	String + base64 detection
Audit logger	Append-only JSONL, SHA-256 chain	JSON + hashlib
HTTP proxy tool	Injects auth headers into requests	httpx (async)
Web login tool	Headless browser form automation	Playwright (Chromium)
Web fetch tool	Authenticated content as markdown	Playwright + readability-lxml
MCP server	Registers tools, manages lifecycle	FastMCP over STDIO
CLI	Credential mgmt outside of Claude	Click

3. Credential lifecycle

3.1 Ingestion (one-time, user-initiated)

1. User runs `krypteia init` — enters master passphrase (masked via `getpass`)
2. Passphrase derived to 32-byte key via PBKDF2-HMAC-SHA256 (salt = SHA256 of service name, 600K iterations per OWASP 2023)
3. Derived key stored in OS keyring (Windows Credential Manager)
4. User runs `krypteia add` — enters credential details (alias, auth_type, username, secret, allowed_urls, allowed_methods)
5. Secret encrypted with AES-256-GCM using unique 12-byte random nonce (`os.urandom`)
6. Encrypted blob persisted to `~/.krypteia/credentials.json`
7. Plaintext secret exists only in process memory during steps 4-6

3.2 Usage — API proxy flow (`krypteia_http_request`)

1. Claude calls `krypteia_http_request(alias, url, method)`
2. Server decrypts the credential from the encrypted store
3. Server checks URL against the credential's allowlist (fail-closed: empty allowlist blocks all)
4. Server checks HTTP method against allowed methods
5. If either check fails: returns error to Claude, logs denial in audit
6. If both pass: injects auth header (Bearer, Basic, or custom)
7. Makes the HTTP request via `httpx`
8. Sanitizes response (scrubs plaintext secret, URL-encoded secret, base64 basic auth)
9. Returns sanitized response to Claude
10. Logs access in audit trail

3.3 Usage — Web login flow (`krypteia_web_login` + `krypteia_web_fetch`)

1. Claude calls `krypteia_web_login(alias, login_url, selectors...)`
2. Server decrypts the credential
3. Playwright launches headless Chromium
4. Browser navigates to login page, waits for network idle
5. Server fills username and password into form fields (CSS selectors)
6. Server clicks submit, waits for navigation
7. Authenticated browser context cached server-side (keyed by alias)
8. Returns { status: "ok", page_title: "..." } to Claude — no credential data
9. Claude calls `krypteia_web_fetch(alias, url)`
10. Server navigates authenticated browser to URL
11. Extracts main content via `readability-lxml`, converts to markdown
12. Sanitizes and returns clean markdown to Claude

4. Encryption details

Property	Value
Algorithm	AES-256-GCM (authenticated encryption)
Key size	256 bits (32 bytes)
Nonce size	96 bits (12 bytes), unique per entry
Nonce generation	os.urandom(12) — CSPRNG
Key derivation	PBKDF2-HMAC-SHA256, 600,000 iterations
KDF salt	First 16 bytes of SHA-256(service_name)
GCM tag	128 bits (included in ciphertext)
Key storage (preferred)	OS keyring
Key storage (fallback)	~/.krypteia/.master-key (0600)

Each credential is encrypted independently with its own nonce. The store uses atomic file writes via .tmp file rename to prevent corruption.

5. Audit system

Append-only JSONL at ~/.krypteia/audit.jsonl. Each event: event_id, event_type, alias, status, details, timestamp, prev_hash, hash. The hash chain links each event to the previous via SHA-256. Modifying any entry breaks the chain. The krypteia audit command verifies integrity.

Never logged: passwords, API keys, tokens, cookies, usernames, encrypted blobs.

6. Threat model

6.1 Threats mitigated

Threat	Mitigation
LLM context leakage	Credentials resolved server-side; only sanitized results returned
Plaintext credential storage	AES-256-GCM encryption at rest with per-entry nonces
Prompt injection exfiltration	URL allowlisting blocks credentials to unauthorized endpoints
Accidental exposure in responses	Sanitizer scrubs secret, URL-encoded, and base64 variants
Credential enumeration by LLM	krypteia_list returns aliases only, never secrets
Audit log tampering	SHA-256 hash chain detects modification

6.2 Known gaps (threats NOT mitigated)

Threat	Notes
Compromised host (admin/root)	Attacker can read process memory
Master key in process memory	No mlock/VirtualLock in v1
Playwright browser memory	Plaintext exists briefly in Chromium during form fill
Malicious MCP client	Client ignoring output boundaries could access raw IPC
User typing password in chat	Cannot prevent direct user input
No rate limiting	Prompt injection could spam tool calls
Master key file fallback	.master-key relies on filesystem perms (weak on Windows)
Deterministic KDF salt	Low risk; passphrase provides entropy
Audit log not signed	Hash chain detects modification, not wholesale deletion
No credential rotation	Manual rotation only, no TTL/expiry
Session cookie management	Browser contexts cached in-memory, no auto timeout

7. Data flow summary

Crosses from Krypteia to Claude (safe)

Credential aliases and metadata, sanitized HTTP response bodies, status codes, web page content as markdown, login status messages, audit events, error messages.

NEVER crosses from Krypteia to Claude

Passwords, API keys, tokens, session cookies, usernames, encrypted blobs, nonces, master key material, raw HTTP auth headers.

8. File layout

- ~/.krypteia/credentials.json — Encrypted credential entries (AES-256-GCM)
- ~/.krypteia/audit.jsonl — Append-only audit log with hash chain
- ~/.krypteia/.master-key — Auto-generated master key (fallback only, 0600)

9. Dependencies

Package	Purpose	Notes
mcp >= 1.0.0	MCP protocol SDK	Anthropic-maintained
cryptography >= 42.0.0	AES-256-GCM	Widely audited, OpenSSL-backed
keyring >= 25.0.0	OS keyring access	Platform-specific backends
httpx >= 0.27.0	Async HTTP client	API proxy
playwright >= 1.40.0	Browser automation	Microsoft-maintained
readability-lxml >= 0.8.1	Content extraction	Mozilla algorithm port
html2text >= 2024.2.26	HTML to markdown	Lightweight
click >= 8.1.0	CLI framework	Pallets project

10. Recommendations for hardening (v2)

- Memory protection** — mlock/VirtualLock to prevent master key swap to disk
- Rate limiting** — Per-alias limits on tool invocations to mitigate prompt injection
- Credential TTL** — Expiry timestamps and rotation reminders
- Session timeout** — Auto-close browser contexts after idle period
- Audit log signing** — Separate signing key to detect wholesale deletion
- MFA/TOTP support** — Handle 2FA challenges during web login
- Credential scoping** — Time-of-day and IP restrictions for API proxy
- Formal security audit** — Third-party penetration testing